



LIONSDEV

Curso de Programação

MÓDULO 10

Deploy com Render

Professor Nicolas Cardoso Motta



Revisando o Módulo 9B



Vitest

describe, it e expect: o teste descreve o que a API deve fazer.



supertest

request(app) testa as rotas sem subir o servidor; por isso o **app.js** fica separado.



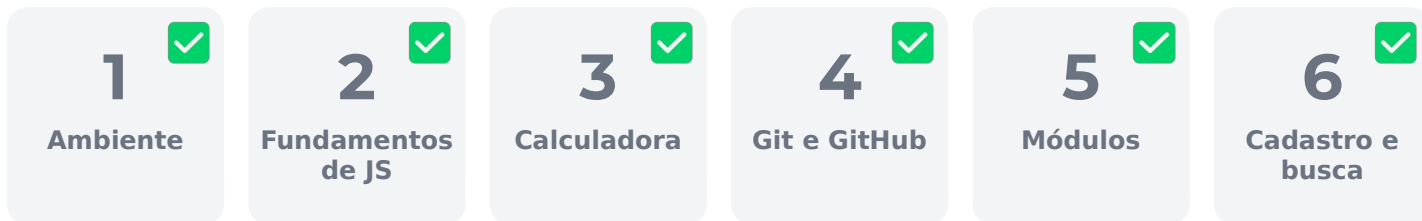
Mocks

vi.mock troca o repository por um dublê: teste rápido e sem banco.

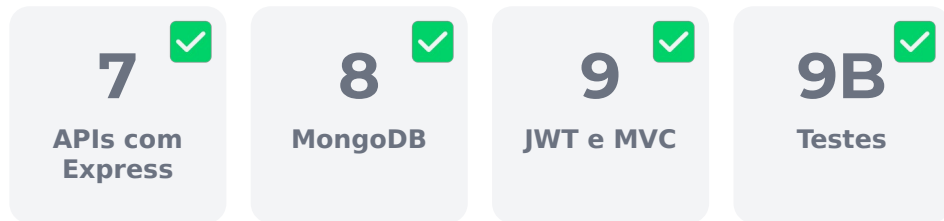
Pergunta para abrir: os testes passam na sua máquina. Isso garante que a API vai funcionar na internet?

Mapa do curso

BASE: JAVASCRIPT, GIT E ORGANIZAÇÃO



BACKEND



FRONTEND E PROJETO



Você está aqui: **Módulo 10**, Deploy com Render

O projeto flashcards



NO MÓDULO 10

A API do flashcards vai para a internet no Render, com o banco no Atlas. A URL pública é o que o app do Módulo 11 vai chamar.



Objetivo

- Entender o que é deploy e o caminho de uma requisição na internet: DNS e HTTPS.
- Preparar a API para produção: porta, variáveis de ambiente e segurança.
- Publicar a API no Render, pelo render.yaml ou pelo painel.
- Automatizar novos deploys a cada push no GitHub.
- Testar a API publicada com o Postman.

**Até agora, as APIs rodavam só no
nosso computador. Para virar um
produto de verdade, falta o próximo
passo: o deploy.**

O que é deploy?



Publicar a aplicação

Colocar a API num ambiente online e estável, fora do seu computador.



Acessível de qualquer lugar

Qualquer cliente, de qualquer lugar, consome a API pela internet.



Disponível 24 horas

Não depende de o seu computador estar ligado.



Deploy contínuo

Cada push no GitHub publica uma versão nova sozinho.

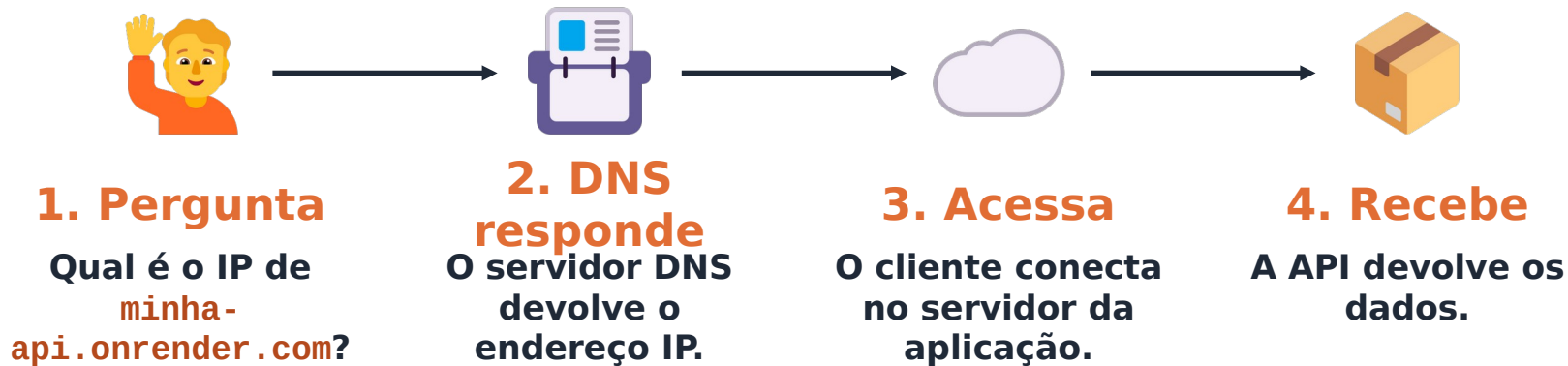


DNS e HTTPS

O caminho até a sua API

Como o DNS funciona

O DNS é a "agenda de contatos" da internet: traduz nomes em endereços IP.



No Render, você ganha um domínio `.onrender.com` com DNS já configurado.

Um caso real: o bloqueio do X

O bloqueio

Em 2024, as operadoras bloquearam os IPs e domínios do X no Brasil, por decisão judicial.

O desvio

O X passou a usar a Cloudflare: o tráfego saía de IPs compartilhados com milhares de outros sites.

A lição

Nomes (DNS) e endereços (IP) são as alavancas que controlam o acesso na internet.

HTTP x HTTPS



HTTP: sem SSL

Os dados trafegam em texto puro.
Quem intercepta a conexão lê
senhas, tokens e tudo o mais.



HTTPS: com SSL/TLS

Os dados vão criptografados.
Quem intercepta vê só um texto
embaralhado.

O Render emite e renova o certificado SSL sozinho: a sua API já nasce em https.

Ambiente local x produção



Local

- Roda em **localhost**, só na sua máquina.
- Variáveis no arquivo **.env**.
- Banco de testes, sem preocupação com escala.



Produção

- URL pública, com HTTPS.
- Variáveis configuradas no painel do Render.
- Banco oficial no Atlas, com dados reais.

Fazer deploy é levar a API do mundo controlado para o mundo real, por isso os cuidados a seguir.

Checklist antes do deploy



Arquitetura

Camadas claras, como no boilerplate.



Credenciais

Senhas com hash e segredos só em variáveis de ambiente.



Acesso

Rotas privadas protegidas com JWT.



Respostas

JSON e status corretos: 200, 201, 400, 401, 404, 500.



Erros

Middleware de erro: a API não cai com uma exceção.



Documentação

README com as rotas e as variáveis necessárias.

Servidores na nuvem: PaaS x IaaS



PaaS

- **Plataforma pronta: você só envia o código.**
- **O provedor cuida de servidor, sistema e escala.**
- **Ex.: Render, Heroku, Vercel.**



IaaS

- **Você recebe uma máquina e configura tudo.**
- **Mais controle, mais trabalho.**
- **Ex.: AWS EC2, Azure VM.**

Como queremos focar no código, e não na infraestrutura, o PaaS é o ideal: por isso o Render.



Preparando a API

A porta vem do Render

package.json

```
"scripts": {  
  "start": "node src/server.js"  
}
```

server.js

```
const PORT = process.env.PORT || 3000  
  
app.listen(PORT, () => {  
  console.log(`Rodando na porta ${PORT}`)  
})
```

O Render define a porta na variável PORT e roda `npm start`. Porta fixa ou script start faltando: o deploy fica no ar, mas a API não responde.

Erros comuns no deploy



Porta fixa

Use `process.env.PORT`.



Variável faltando

`MONGO_URI` e `JWT_SECRET` configuradas no Render.



Atlas bloqueando

O Render usa IPs variáveis: libere `0.0.0.0/0` no Network Access.



Maiúsculas no import

`./Models/User.js` funciona no Windows e quebra no Linux do Render.



Pasta errada

O `package.json` precisa estar na raiz do repositório (ou configure o Root Directory).



node_modules no Git

O Render instala tudo com `npm install`: deixe no `.gitignore`.

Deu erro? Os logs do Render dizem exatamente onde: leia de baixo para cima.

Scripts: dev e start

package.json

```
"scripts": {  
  "dev": "node --watch src/server.js",  
  "start": "node src/server.js"  
}
```

No terminal

```
$ npm run dev      # no seu computador  
$ npm start        # o que o Render roda
```

O dev reinicia a cada mudança, para desenvolver. O start é o comando de produção. O start não precisa do "run".

Uma rota de saúde

```
app.get('/health', (req, res) => {  
  res.status(200).json({ status: 'ok' })  
})
```

No Render, informe /health no Health Check Path: ele confere se a API está respondendo e só troca a versão no ar quando a nova estiver saudável.



Deploy no Render

O fluxo de um deploy

Do push ao ar, o Render cuida de tudo:



1. git push

O código vai para o GitHub.

2. Detecta

O Render percebe o commit novo.

3. Build

Roda `npm install`.

4. Start

Roda `npm start`.

5. Live

API no ar, em HTTPS.

Opção 1: o render.yaml

```
services:  
  - type: web  
    name: api-banco-digital  
    runtime: node  
    plan: free  
    buildCommand: "npm install"  
    startCommand: "npm start"  
    autoDeploy: true  
    envVars:  
      - key: MONGO_URI  
        sync: false  
      - key: JWT_SECRET  
        generateValue: true
```

infra como código → em vez de clicar no painel, você descreve o serviço num arquivo versionado.

build / start → os comandos para instalar e iniciar a API.

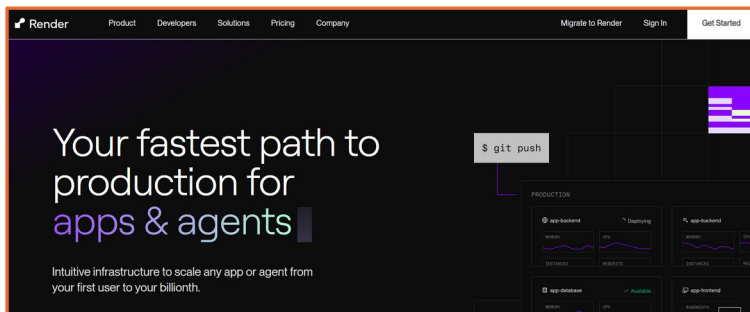
sync: false → o valor não fica no arquivo: o Render pergunta no painel. Ideal para a **MONGO_URI**.

generateValue → o Render gera um segredo aleatório, perfeito para o **JWT_SECRET**.

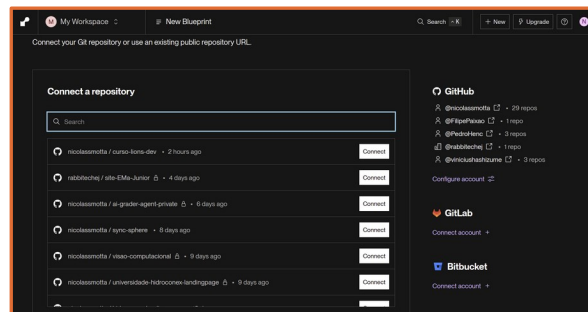
Publicando com Blueprint

- 1 **Suba o projeto para o GitHub**
Com o `render.yaml` na raiz.
- 2 **Entre no Render com o GitHub**
Assim ele enxerga os seus repositórios.
- 3 **New > Blueprint**
Escolha o repositório: o Render lê o `render.yaml`.
- 4 **Preencha a MONGO_URI e faça o deploy**
Acompanhe o build pelos logs até aparecer "Live".

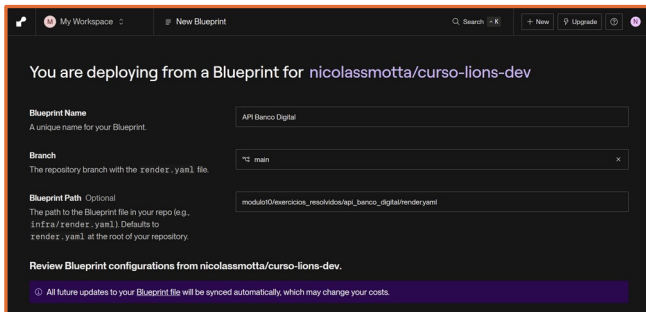
Blueprint passo a passo



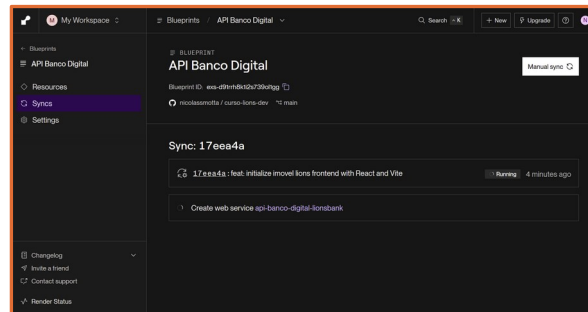
Acesse o Render



Crie um Blueprint



Configure a MONGO_URI

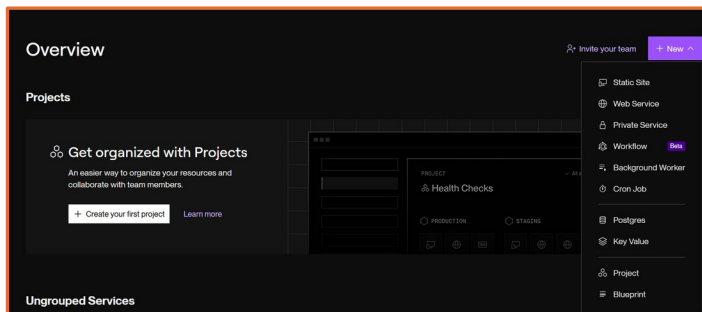


Deploy subindo

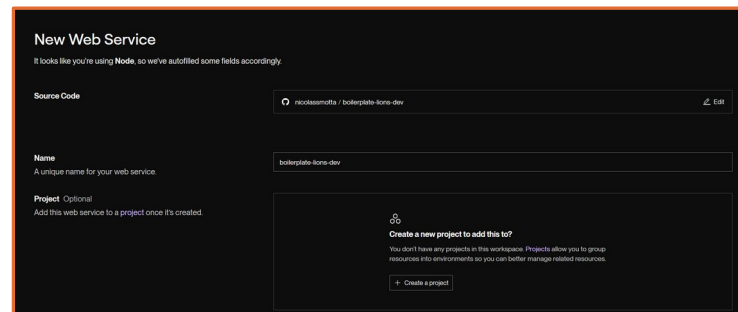
Opção 2: configuração manual

- 1 **New > Web Service**
Conecte o repositório do GitHub.
- 2 **Definições iniciais**
Nome, região e branch.
- 3 **Linguagem e comandos**
Node, build `npm install` e start `npm start`.
- 4 **Plano e variáveis**
Plano Free e as variáveis de ambiente, uma a uma.

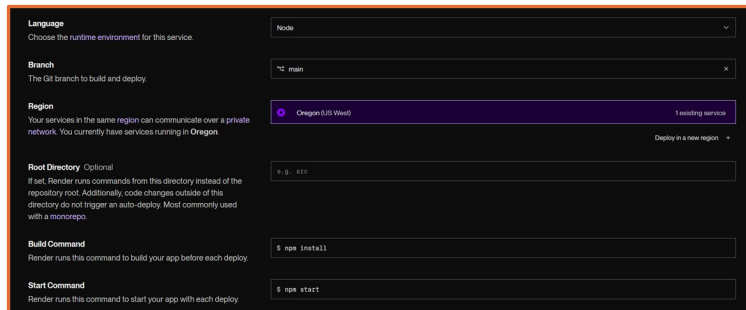
Configuração manual passo a passo



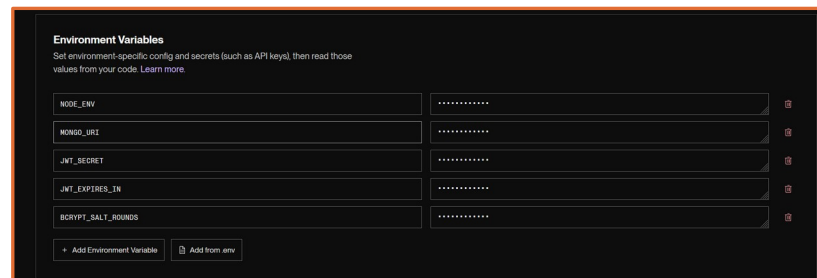
Crie um Web Service



Definições iniciais



Linguagem e comandos



Variáveis de ambiente



Cuidado: plano free e cold start

No plano gratuito, o serviço **dorme depois de uns 15 minutos sem requisições.**

- **A primeira requisição depois disso acorda o serviço e pode levar perto de um minuto.**
- **Não é erro: espere e tente de novo. Antes de uma apresentação, faça uma requisição para "acordar" a API.**

Segurança em produção



helmet

Um middleware que adiciona headers de segurança às respostas.

```
app.use(helmet())
```



Rate limit

Limita requisições por IP: dificulta ataques de força bruta no login.

```
app.use(rateLimit({  
  windowMs: 15 * 60 * 1000,  
  limit: 100,  
}))
```

Pacotes helmet e express-rate-limit. Com a API pública na internet, qualquer um pode tentar acessá-la.

Para ir além



Domínio próprio

Troque o **.onrender.com** pelo seu domínio, com SSL automático.



Rollback

O Render guarda o histórico: volte para um deploy anterior em um clique.



Logs e health check

Acompanhe a execução e deixe o Render verificar se a API está viva.

Atividade Prática 01

Deploy

Sua API no ar: publique o boilerplate no Render.

- 1. Garanta o `process.env.PORT` no `server.js` e o script `start` no `package.json`.**
- 2. Suba o projeto para o GitHub, com o `.env` no `.gitignore`.**
- 3. Publique pelo Blueprint ou pelo Web Service, configurando `MONGO_URI` e `JWT_SECRET`.**
- 4. Acesse a URL pública e confira a rota `/`.**

Atividade Prática 02

Deploy

Sua API no Postman: publique um exercício que você já fez.

- 1. Escolha uma das APIs dos módulos 8 ou 9.**
- 2. Publique no Render do mesmo jeito.**
- 3. No Postman, troque `http://localhost:3000` pela URL do Render e teste todas as rotas.**



Dica: crie uma variável `baseUrl` no Postman: trocar entre local e produção vira um clique.

Lista de exercícios



Deploy e variáveis de ambiente
process.env, porta dinâmica, .env e o script start.



API de Banco Digital
Um projeto completo, pronto para publicar no Render.

No repositório: [modulo10/lista_de_exercicios/](#)

Desafio: flashcards no ar

Fio condutor

Publique no Render a API do flashcards (a versão do boilerplate, com login).

- Confira a porta de `process.env.PORT`, o script `start` e a rota `/health`.
- No Render, cadastre `MONGO_URI` e `JWT_SECRET` nas variáveis de ambiente.
- No Atlas, libere o acesso de rede para o Render.
- Teste pelo Postman na URL pública: cadastro, login e GET `/api/baralhos` com o token.
- Coloque a URL pública no README do repositório.



Cuidado: o `.env` nunca vai para o GitHub: confira o `.gitignore` antes do push.



Desafio: rode `npm test` antes de cada deploy e só publique se tudo passar.

Material do módulo



Aponte a câmera do celular

[github.com/nicolassmotta/curso-lions-dev/
tree/main/modulo10](https://github.com/nicolassmotta/curso-lions-dev/tree/main/modulo10)



conteudo/
teoria comentada e o cheat sheet do módulo



lista_de_exercicios/
listas para praticar em casa



exercicios_resolvidos/
gabaritos, para conferir depois de tentar



Fim!

No Módulo 11, a sua API ganha uma interface.